In [ ]:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Load MNIST
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# Normalize
x_train = x_train.astype("float32") / 255
x_test = x_test.astype("float32") / 255

# Flatten (for dense models)
x_train_flat = x_train.reshape(-1, 28*28)
x_test_flat = x_test.reshape(-1, 28*28)

print(x_train.shape, y_train.shape)

def build_dense_model():
    model = keras.Sequential([
        layers.Dense(128, activation='relu', input_shape=(784,)),
        layers.Dense(64, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    return model

model = build_dense_model()

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    x_train_flat, y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.1
)

model.evaluate(x_test_flat, y_test)

optimizers = ['adam', 'sgd', 'rmsprop']

for opt in optimizers:
    print(f"\nTraining with optimizer: {opt}")

    model = build_dense_model()
    model.compile(
        optimizer=opt,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    model.fit(x_train_flat, y_train, epochs=3, batch_size=32, verbose=0)
    loss, acc = model.evaluate(x_test_flat, y_test, verbose=0)

    print(f"{opt} accuracy: {acc:.4f}")


def build_deep_model():
    model = keras.Sequential([
        layers.Dense(256, activation='relu', input_shape=(784,)),
        layers.Dense(128, activation='relu'),
        layers.Dense(64, activation='relu'),
        layers.Dense(32, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    return model

def build_dropout_model():
    model = keras.Sequential([
        layers.Dense(128, activation='relu', input_shape=(784,)),
        layers.Dropout(0.3),
        layers.Dense(64, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(10, activation='softmax')
    ])
    return model

# Reshape for CNN
x_train_cnn = x_train[..., None]
x_test_cnn = x_test[..., None]

def build_cnn():
    model = keras.Sequential([
        layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
        layers.MaxPooling2D((2,2)),
```

```
        layers.Conv2D(64, (3,3), activation='relu'),
        layers.MaxPooling2D((2,2)),
        layers.Flatten(),
        layers.Dense(64, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    return model

model = build_cnn()

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(x_train_cnn, y_train, epochs=5, validation_split=0.1)
model.evaluate(x_test_cnn, y_test)

optimizer = keras.optimizers.Adam(learning_rate=0.0005)

model.compile(
    optimizer=optimizer,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

results = []

for opt in ['adam', 'sgd']:
    for layers_num in [2, 4]:
        model = keras.Sequential()

        model.add(layers.Dense(128, activation='relu', input_shape=(784,)))

        for _ in range(layers_num - 1):
            model.add(layers.Dense(64, activation='relu'))

        model.add(layers.Dense(10, activation='softmax'))

        model.compile(optimizer=opt,
                      loss='sparse_categorical_crossentropy',
                      metrics=['accuracy'])

        model.fit(x_train_flat, y_train, epochs=3, verbose=0)
        _, acc = model.evaluate(x_test_flat, y_test, verbose=0)

        results.append((opt, layers_num, acc))

print(results)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
(60000, 28, 28) (60000,)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9263 - loss: 0.2553 - val_accuracy: 0.9650 - val_loss: 0.1208
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.9669 - loss: 0.1086 - val_accuracy: 0.9718 - val_loss: 0.0976
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9767 - loss: 0.0759 - val_accuracy: 0.9785 - val_loss: 0.0787
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9818 - loss: 0.0562 - val_accuracy: 0.9690 - val_loss: 0.1106
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9854 - loss: 0.0455 - val_accuracy: 0.9720 - val_loss: 0.0966
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9675 - loss: 0.1071

Training with optimizer: adam
adam accuracy: 0.9753

Training with optimizer: sgd
```